
Technical Report — Brand Feedback Analyzer Project

Overview

This project is an end-to-end sentiment analysis and recommendation system focused on Egyptian brands, with a strong implementation path for the telecom sector. It combines data collection, text preprocessing, machine learning classification, company ranking, and an interactive Streamlit dashboard in a single workflow.

The application supports two operating modes: Reddit-based collection for brand mentions and a telecom review dataset path for preloaded market analysis and recommendation views. The final system predicts sentiment labels, summarizes brand-level results, and ranks companies using a recommendation score that blends sentiment, engagement, and confidence.

Objectives

The technical goal of the system is to ingest unstructured user feedback, transform it into normalized textual features, classify each record into positive, neutral, or negative sentiment, and aggregate the output into business-facing comparison metrics. The product goal is to provide a dashboard that helps users compare telecom operators, inspect sentiment patterns, and identify the highest-ranked company based on model outputs and engagement indicators.

System Architecture

The project is organized around several core modules.

Module	Main role	Key implementation details
Data collection	Gather Reddit posts and comments	Query generation, Reddit JSON fetching, comment retrieval, duplicate removal, Egypt relevance filtering.
Training	Build the sentiment classifier	Reads labeled data, applies preprocessing, vectorizes text with TF-IDF, trains a LinearSVC model, and saves artifacts.
Inference pipeline	Predict on new feedback	Loads svmmodel.pkl and vectorizer.pkl, preprocesses new text, predicts sentiment, and writes processed CSV files.
Recommendation engine	Rank companies	Computes sentiment index, engagement index, confidence index, and final recommendation score.
Streamlit application	User interface and analytics	Displays dashboards, comparison views, sentiment charts, recommendation results, and MLOps-style monitoring widgets.

The Streamlit layer acts as the integration point between trained artifacts, processed datasets, ranking logic, and visualization outputs. The code also includes a pipeline-stage snapshot mechanism that reports collection, cleaning, feature engineering, training, evaluation, deployment, and monitoring status for the application workflow.

Data Sources

The model training pipeline uses a labeled dataset stored at `data/labeled/TwitterData.csv` with sentiment labels mapped from numeric categories into negative, neutral, and positive classes. The training notebook shows the underlying dataset contained 162,980 rows initially, with 162,968 usable rows after cleaning and removal of missing or empty values.

At inference time, the application supports two feedback sources. One path collects Reddit posts and comments related to Egyptian brands using brand-specific and Arabic-aware query expansion, while the other path loads a telecom review dataset and maps its schema into the application's internal review format before prediction.

Data Collection and Filtering

The Reddit scraping logic builds multiple brand-centered search queries such as brand name, Egypt-specific variants, and Arabic aliases when applicable. It retrieves both posts and linked comments, stores metadata such as score, number of comments, permalink, and timestamp, and removes duplicates using record type, post ID, and text.

A relevance filter is used to keep Egypt-related content by checking brand terms together with Egypt-related keywords such as location or country references. If the Egypt-specific search returns no results, the pipeline expands to a broader global Reddit search as a fallback path.

The Streamlit version adds improved credential handling for Reddit OAuth, including optional use of `.streamlit/secrets.toml` or environment variables, token reuse, retry logic, rate-limit handling, and clearer error reporting when Reddit returns 403 or 429 responses. This makes the app-side collection logic more robust than the minimal CLI version.

Text Preprocessing

The preprocessing pipeline is intentionally lightweight and rule-based. Text is lowercased, HTML tags and URLs are removed, Arabic diacritics are stripped, punctuation and symbols are removed, and whitespace is normalized before classification.

Tokenization is implemented as whitespace splitting, followed by a simple custom lemmatization and stemming stage. The rule set includes basic English suffix handling such as sses, ies, and trailing s, plus a simple Arabic prefix trimming rule for longer tokens.

During inference, preprocessing optionally preserves stemmed tokens only when they appear in the trained vectorizer vocabulary, which helps align runtime tokens with learned TF-IDF features. This creates consistency between training-time and inference-time representations.

Model Development

The training script uses `train_test_split` with stratification and a test size of 20 percent to preserve class balance during evaluation. Text is transformed using `TfidfVectorizer(max_features=5000)`, and the chosen classifier is `LinearSVC(random_state=42)`.

The notebook documents comparison experiments across three baseline classifiers: Logistic Regression, Multinomial Naive Bayes, and Linear SVM. Linear SVM achieved the best performance and was therefore selected as the deployed model artifact.

Model comparison

Model	Accuracy	Notes
Logistic Regression	0.9267	Strong baseline with balanced performance across classes.
Naive Bayes	0.7426	Noticeably weaker, especially on negative recall.
Linear SVM	0.9421	Best overall accuracy and strongest weighted performance; selected for deployment.

Evaluation Results

The notebook reports the following class-level metrics for the selected Linear SVM model on a 32,594-record test split.

Class	Precision	Recall	F1-score	Support
Negative	0.92	0.87	0.90	7,102
Neutral	0.95	0.98	0.96	11,042
Positive	0.95	0.95	0.95	14,450
Weighted average	0.94	0.94	0.94	32,594

The Streamlit monitoring function also recomputes evaluation metrics from the labeled dataset and saved model artifacts, returning accuracy, precision, recall, F1, loss, and evaluation sample size for dashboard

display. This means model-quality indicators are not hardcoded into the UI; they are derived from current artifacts when available.

Saved Artifacts

The training pipeline persists two production artifacts: `models/svmmodel.pkl` for the classifier and `models/vectorizer.pkl` for the TF-IDF representation. These files are loaded by both the CLI-style inference path and the Streamlit application.

Processed output files are stored in `data/processed` using a `brand_predicted.csv` naming pattern, while raw collected data is stored in `data/raw` using a `brand_raw.csv` pattern. The recommendation loader scans the processed directory and builds cross-company ranking tables from all matching predicted files.

Recommendation Engine

The recommendation layer converts record-level sentiment predictions into brand-level profiles. For each company, it computes counts and percentages of positive, neutral, and negative feedback, average score, total engagement, sentiment index, and confidence index.

The sentiment index is derived from weighted sentiment percentages using the weights positive = 1.0, neutral = 0.35, and negative = -0.65. Engagement is computed from post score and number of comments, while confidence is tied to volume and saturates at 50 records.

The final recommendation score combines three components: 65 percent sentiment index, 20 percent engagement index, and 15 percent confidence index. Companies are then sorted by recommendation score, followed by positive percentage, negative percentage, and total record count to break ties.

Dashboard and Visual Analytics

The Streamlit application is designed as a broad analytics interface rather than a single prediction form. It includes summary cards, sentiment donuts, bar charts, subreddit distributions, sentiment-by-record-type plots, score-versus-sentiment scatter plots, timeline charts, and telecom-specific rating, city, service-type, and customer-type visualizations.

The application also contains a dedicated recommendation page showing the best company, recommendation score, positive and negative rates, and a ranking table for the top companies. A separate MLOps workflow page displays pipeline stages, dataset overview, feature count, model classes, model file size, saved timestamp, deployment status, pseudo-latency, and a live-style training logs console.

Telecom Workflow

The application includes a predefined telecom mode that loads a telecom review CSV, maps several brand aliases such as Vodafone, Orange, Etisalat, and WE into canonical company names, and predicts sentiment using the trained SVM model. The initialization routine precomputes cached processed CSV files for the four telecom operators and uses them to populate the recommendation system at startup.

This telecom-specific path allows the dashboard to demonstrate comparative market analysis even when live Reddit collection is unavailable or restricted. It also provides a structured evaluation context for recommendation ranking because the four operators are normalized into a shared schema before prediction and aggregation.

Operational Flow

From a systems perspective, the project follows a six-stage pattern: data collection, data cleaning, feature engineering, model training, model evaluation, and deployment, followed by monitoring in the dashboard layer. The CLI pipeline variant performs collection, preprocessing, prediction, summary generation, CSV export, and ranking in one executable flow for a given brand.

The Streamlit layer extends this by adding caching, on-demand re-analysis, cached file reuse, multi-brand comparison, and runtime MLOps-style introspection. This gives the project both a script-oriented execution path and an interactive analytics path.

Strengths

- The system covers the full lifecycle from raw text ingestion to ranked company recommendations.
- Linear SVM achieved the strongest measured accuracy among tested baselines in the training notebook.
- The application supports both live-style data acquisition and dataset-based fallback analysis paths.
- The recommendation layer is interpretable because it exposes sentiment, engagement, and confidence components separately.
- The Streamlit dashboard includes operational and model-quality visibility rather than only final predictions.

Conclusion

The project is a solid end-to-end applied machine learning system for sentiment-driven company comparison, combining data ingestion, classical NLP, model training, prediction, ranking, and dashboard delivery in a coherent workflow. Its strongest technical result is the deployed Linear SVM model, which outperformed the tested Logistic Regression and Naive Bayes baselines in the documented notebook experiments.